

HackShield Club

Web Exploitation

Webinar Documentation

Beginner Friendly | Technically Complete | CTF Ready

Property	Details
Domain	Web Exploitation
Week	Week 2 of 6
Format	Introductory Webinar + 7-Day Program
Level	Beginner to Intermediate
Prerequisites	Basic Linux, HTTP awareness (Week 1)
Topics Covered	HTML/HTTP, SQLi, XSS, Auth Flaws, File Upload, Burp Suite, CTF

Introduction

What is Web Exploitation?

Web exploitation is the practice of identifying and taking advantage of security weaknesses in web applications. Every website you visit runs code, stores data, processes user input, and communicates with servers — and each of those steps can contain a flaw that an attacker can exploit.

As a security professional or ethical hacker, understanding web exploitation helps you think like an attacker so you can defend applications better, find vulnerabilities before malicious actors do, and participate in bug bounty programs and CTF competitions.

Why Web Exploitation Matters

Web applications are the most targeted surface in modern cybersecurity. According to industry reports, over 75% of cyberattacks target the application layer — not the network layer. This is because:

- Web apps are internet-facing and accessible to anyone
- They handle sensitive data — passwords, payments, personal information
- Developers often prioritize speed over security
- A single vulnerability can affect thousands or millions of users

The Attacker's Mindset

Thinking like an attacker does not mean acting maliciously. It means asking: 'What would happen if I sent unexpected input here?' or 'What if I modified this request?' or 'Is the server really checking my identity, or just trusting what I send?'

Every vulnerability we study this week comes down to one root cause: the application trusted user-supplied input without properly validating it.

KEY INSIGHT

The core of almost every web vulnerability: the server trusted data from the client without verifying it. Your job as a tester is to find where that trust is misplaced.

Legal and Ethical Boundaries

All techniques taught in this program must only be used on:

- Systems you own or have built yourself
- Deliberately vulnerable practice apps (DVWA, WebGoat, HackTheBox, TryHackMe)
- Systems where you have explicit written permission
- CTF competition environments during the competition window

WARNING

Exploiting systems without permission is illegal under the Computer Fraud and Abuse Act (CFAA) and equivalent laws worldwide. Unauthorized access, even for 'educational' purposes, can result in criminal prosecution. Always practice in legal environments.

Recommended Practice Environments

Platform	Best Used For
DVWA (Damn Vulnerable Web App)	Local practice — SQLi, XSS, File Upload, Auth
PortSwigger Web Security Academy	Structured labs for every topic in this webinar
TryHackMe	Guided rooms, beginner friendly
HackTheBox	Realistic machines, intermediate level
PicoCTF	CTF-style beginner challenges

Web Fundamentals — How the Web Works

Before exploiting anything, you must deeply understand how normal web communication works. Every attack is just a twist on normal web behaviour.

1.1 The Client-Server Model

Every web interaction follows the same pattern: a client (your browser) sends a request, and a server returns a response.

Component	Role
Browser (Client)	Renders pages, executes JavaScript, sends HTTP requests
Web Server	Receives requests, runs application logic, returns responses
Database	Stores persistent data — users, posts, products, orders
DNS	Translates domain names (google.com) to IP addresses

1.2 HTTP — HyperText Transfer Protocol

HTTP is the language browsers and servers use to communicate. Every click, form submission, and page load is an HTTP request.

HTTP Request Structure

A raw HTTP request looks like this:

```
GET /login HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Cookie: session=abc123

(empty line marks end of headers, body follows for POST)
```

HTTP Methods

Method	Purpose and Security Relevance
GET	Retrieve data. Parameters visible in URL. Never use for sensitive data.

POST	Submit data in request body. Used for forms, logins, uploads.
PUT	Update a resource. Often misconfigured — may allow unauthorized updates.
DELETE	Remove a resource. Should require authentication.
OPTIONS	Ask server what methods are allowed. Useful for recon.

HTTP Status Codes

Code	Meaning and What It Tells an Attacker
200 OK	Request succeeded — content returned
301/302	Redirect — follow to find the real endpoint
400	Bad Request — your input was malformed
401	Unauthorized — authentication required
403	Forbidden — authenticated but not allowed (try privilege escalation)
404	Not Found — resource does not exist
500	Internal Server Error — your input may have broken something (useful for SQLi/bugs)

1.3 HTML and Forms

HTML forms are the primary way users send data to servers — and the primary attack surface for many vulnerabilities.

```
<form action="/login" method="POST">
  <input type="text"      name="username" />
  <input type="password" name="password" />
  <input type="hidden"   name="csrf_token" value="abc123" />
  <button type="submit">Login</button>
</form>
```

Key things to observe in any form:

- action — where the data is submitted to
- method — GET or POST
- hidden fields — data sent silently, often contains tokens or flags
- input names — these become the parameter names in the request

1.4 Cookies and Sessions

HTTP is stateless — by default the server does not remember who you are between requests. Cookies solve this by storing a session identifier in your browser.

Cookie Attribute	Security Meaning
HttpOnly	JavaScript cannot read this cookie — prevents XSS theft
Secure	Only sent over HTTPS — prevents interception
SameSite	Controls cross-site sending — prevents CSRF
Expires/Max-Age	When cookie expires — session vs persistent

ATTACKER PERSPECTIVE

If a cookie lacks HttpOnly, it can be stolen via XSS. If it lacks Secure, it can be intercepted on HTTP. If it lacks SameSite, it may be vulnerable to CSRF. Always check cookie attributes during recon.

1.5 Browser Developer Tools

DevTools is your most important tool for web recon. Open with F12 in any browser.

Tab	What to Look For
Elements	Hidden form fields, comments in HTML, page structure
Network	All HTTP requests/responses, headers, cookies, payloads
Console	JavaScript errors, debug output left by developers
Application	Cookies, LocalStorage, SessionStorage — stored tokens
Sources	JavaScript source files — may contain secrets or logic

1.6 Day 1 Task — Analyze a Webpage

Using only your browser DevTools on a provided target page:

1. Open DevTools (F12) and go to the Network tab
2. Reload the page and observe all requests made
3. Find any hidden input fields in the HTML source
4. Identify what cookies are set and check their attributes
5. Look in the JavaScript console for any debug output
6. Find the flag hidden somewhere in the page structure

HINT

Flags in web challenges are often hidden in HTML comments (`<!-- flag{...} -->`),

hidden input fields, HTTP response headers, or JavaScript source files. Check all of these.